

计算机通信与网络大作业

—三种 ARQ 协议编程实现报告

实验目的：

1. 理解三种 ARQ 协议（停止等待、回退 N 帧、选择性重传）的工作原理。
2. 掌握在模拟环境中实现 ARQ 协议的方法。
3. 通过模拟丢帧情况，观察不同 ARQ 协议在丢帧时的处理机制和性能差异。
4. 分析比较三种 ARQ 协议的优缺点及适用场景。

实验内容：

请借助 AI 大模型，可仿照课件案例动画，编程实现停止等待 ARQ(10 分)、返回 N 帧 ARQ (40 分)和选择性重复 ARQ 算法(50 分)，要求在手工删除数据帧或确认帧(即模拟丢帧)时算法能正确处理丢帧的情况。

最终算法不依赖于环境，提交可执行文件、源程序、课程报告。

实验过程：

我使用了惯用的 C 语言实现 ARQ 协议的模拟。不过，C 语言没有内置的图形库，所以我在 Visual Studio 上安装了 Raylib 图形库来模拟 ARQ 协议的行为，展示发送和接收的过程，并允许用户通过输入命令来模拟丢帧。

由于将三种 ARQ 协议的模拟放在一个程序中较为复杂，我选择将其拆开，每个 ARQ 协议的模拟由一个完整 C 语言程序进行实现。

使用状态机管理发送方和接收方的状态，使用结构体数组管理帧和 ACK，使用定时器（基于帧计数）实现超时。

Raylib 图形库不能识别中文，输入中文会变为“？”，必须使用英文书写信息。

在高版本 Raylib 中，'DrawRectangleRoundedLines' 函数有 5 个参数，但实际操作中发现只能调用 4 个参数 (Rectangle, float, int, Color)，需要删除第 4 个参数，即线宽。

使用'strcpy'函数需要在文件开头定义：`#define _CRT_SECURE_NO_WARNINGS`

（1）停止等待 ARQ

特点：发送方每发送一帧就等待确认
收到确认后再发送下一帧
超时未收到确认则重发
简单但效率较低

预期结果：

发送方发送帧 0

帧 0 移动到接收方

接收方接收帧 0 并发送 ACK0
ACK0 返回到发送方
发送方接收 ACK0 发送帧 1
重复上述过程

数据帧丢失场景：
开启"丢帧模式"
当帧 0 出现时，点击它手动丢弃
预期结果：
帧 0 消失（标记为丢失）
等待超时后（约 3 秒）
发送方重新发送帧 0
正常传输恢复

ACK 丢失场景：
开启"丢帧模式"
当 ACK0 出现时，点击它手动丢弃
预期结果：
ACK0 消失（标记为丢失）
等待超时后（约 3 秒）
发送方重新发送帧 0
接收方重新发送 ACK0

由预期结果设计：

1. 定义帧的结构，包括位置、大小、状态（是否发送、是否接收、是否丢失等）、类型（数据帧或 ACK 帧）和序号。
2. 在窗口中绘制发送方和接收方，以及帧的移动过程。
3. 实现帧的移动：从发送方移动到接收方，然后 ACK 从接收方移动到发送方。
4. 实现点击丢弃功能：在丢帧模式下，点击帧则将其标记为丢失，并使其消失。
5. 超时处理：设置定时器，当发送方发送一帧后，如果在规定时间内没有收到 ACK，则重传。

注意：停止等待 ARQ 协议中，发送方发送一帧后必须接收该帧的 ACK 后才能发送下一帧。

主要步骤：

1. 初始化窗口。
数据帧：active=false, lost=false
ACK 帧：active=false, lost=false
currentSeq = 0
receiverSeq = 0
waitingForAck = false

发送方发送数据帧：

设置 dataFrame 的 active 为 true，位置在发送方，目标位置是接收方（移动速度指向接收方）

设置 `dataFrame` 的 `seq` 为 `currentSeq`

启动定时器（记录发送时间）

`waitingForAck = true`

接收方接收到数据帧（当数据帧到达接收方区域）：

如果数据帧的 `seq` 等于 `receiverSeq`，则接收（打印接收信息），然后发送 ACK（`ackFrame` 激活，从接收方位置向发送方移动，`ackFrame.seq = receiverSeq`）

然后 `receiverSeq = 1 - receiverSeq`（准备接收下一帧）

并将数据帧设为非激活（到达后消失）

发送方接收到 ACK 帧（当 ACK 帧到达发送方区域）：

如果 ACK 帧的 `seq` 等于 `currentSeq`，则说明正确接收，则：

`currentSeq = 1 - currentSeq`（准备发送下一帧）

`waitingForAck = false`

停止定时器

ACK 帧设为非激活

可以发送下一帧

注意：如果 ACK 帧序号不对，则丢弃（但停止等待 ARQ 中不会出现乱序，因为只有一帧）

2. 定义帧结构（`Frame`），包含位置、大小、颜色、状态（是否激活）、类型（数据帧或 ACK 帧）、序号、移动速度、是否丢失等。

我用一个结构体表示帧（包括数据帧和 ACK 帧）：

```
typedef struct {  
    Rectangle rect;  
    Color color;  
    bool active; // 是否正在传输  
    bool isACK; // true 表示 ACK，false 表示数据帧  
    int seq; // 序号：0 或 1  
    Vector2 speed; // 移动速度  
    bool lost; // 是否被丢弃（丢失）  
} Frame;
```

3. 定义发送方和接收方的位置（矩形区域）。

4. 使用两个 `Frame` 实例：一个用于当前数据帧，一个用于当前 ACK 帧（因为同一时刻最多只有一个数据帧和一个 ACK 帧在传输）。

但是，当数据帧丢失时需要重传，所以发送方需要保存当前帧的数据（重传时使用同一个帧结构，重置位置并重新激活）。

因此设计：

`Frame dataFrame;` // 当前传输的数据帧（可以是 0 或 1）

`Frame ackFrame;` // 当前传输的 ACK 帧（可以是 0 或 1）

状态变量：

`int senderState;` // 发送方状态：0 表示可以发送（或重传）当前帧，1 表示已发送当前帧，等待 ACK

`int currentSeq;` // 发送方当前应该发送的帧序号（0 或 1）

`int receiverSeq;` // 接收方期望的帧序号（0 或 1）

`bool waitingForAck;` // 发送方是否正在等待 ACK

`bool ackReceived;` // 在等待 ACK 期间，是否收到 ACK

5. 状态：

发送方状态：可以发送帧 0，等待 ACK0；收到 ACK0 后发送帧 1，等待 ACK1；如此往复。

接收方状态：等待帧 0，收到后发送 ACK0；然后等待帧 1，收到后发送 ACK1。

6. 丢帧模式：

一个布尔变量，当按下某个键（比如 F 键）时，丢帧模式切换。

在丢帧模式下，如果鼠标点击了正在传输的帧（数据帧或 ACK 帧），则设置该帧的 `lost` 为 `true`，并取消激活（这样它就不会被绘制和移动）。同时，模拟帧丢失。

注意：帧的移动通过更新帧的位置（每帧加上速度）来实现。

7. 定时器：发送方发送一帧后启动定时器（记录发送时间），如果超过 3 秒没有收到 ACK，则重发当前帧。

超时处理：

如果 `waitingForAck` 为 `true`，且当前时间减去发送时间大于 3000 毫秒，则触发重传：重传当前数据帧（重置 `dataFrame` 的位置到发送方，激活，`lost` 重置为 `false`，并重新启动定时器）

8. 绘制：

绘制发送方和接收方区域（矩形）

绘制数据帧和 ACK 帧（如果 `active` 且没有 `lost`）

绘制状态文本（例如：当前发送方状态，接收方状态，丢帧模式是否开启等）

9. 更新：帧的移动，当帧到达接收方时，接收方发送对应的 ACK；当 ACK 到达发送方时，发送方发送下一帧（或重传的帧到达则发送 ACK）。

注意：重传时，发送方不会发送新帧，而是重传当前帧。

(2) 返回 N 帧 ARQ

特点:

- 发送方维护一个发送窗口
- 接收方只按序接收帧
- 超时后重发所有未确认帧
- 效率高于停止等待

预期结果:

- 点击"开始模拟"按钮
- 依次发送 4 个帧 (帧 0-3)
- 接收方按序接收并依次发送对应的 ACK (0-3)

中间帧丢失场景:

- 开启"丢帧模式"
- 当帧 1 出现时, 点击它手动丢弃

预期结果:

- 帧 1 消失
- 接收方接受帧 0 并发送 ACK0, 但是丢弃后续帧 (帧 2-3 也不发送 ACK)
- 等待超时
- 发送方重新依次传帧 1-3
- 接收方按序接收并发送 ACK1-3

由预期结果设计:

1. 定义帧结构体, 包含位置、大小、序号、是否激活、是否丢失、是否被确认、移动速度等。
2. 定义 ACK 结构体, 类似帧。
3. 定义按钮 (开始模拟、丢帧模式开关)。
4. 状态变量: 模拟是否开始、丢帧模式是否开启、发送窗口、接收方期望序号、超时计时器等。

实现步骤:

1. 初始化窗口和变量。
2. 绘制:
 - 绘制发送方和接收方区域
 - 绘制每个帧 (如果未丢失且激活)
 - 绘制每个 ACK (如果激活且未丢失)
 - 绘制按钮和状态信息
3. 当点击开始按钮, 初始化 4 个帧, `frames[0-3]`: 激活, 位置在发送方, 垂直排列 (`y` 坐标递增), `x` 坐标相同 (发送方起始位置)。
`base = 0;`
设置一个计时器 `timer`, 从 0 开始, 当有未确认帧时开始计时 (即 `base < 4` 时)。

`expectedSeq = 0;` // 接收方期望的帧序号

4. 在每一帧更新中:

更新帧的位置 (向接收方移动), 如果帧到达接收方且未丢失, 检查帧序号是否是接收方期望的序号 (`expectedSeq`):

如果是, 则激活对应的 ACK (ACK 的序号等于该帧序号), 并增加 `expectedSeq`。

如果不是, 则丢弃该帧 (标记为丢失, 且不发送 ACK)。

更新 ACK 的位置 (向发送方移动)。如果 ACK 到达发送方, 则标记该 ACK 对应的帧为已确认, 并更新发送窗口

5. 超时处理: 为发送窗口中的第一个未确认帧设置计时器。如果超时 (3 秒), 则重传该帧及其后续所有已发送但未确认的帧 (重置它们的位置到发送方, 并重置丢失状态, 重新开始移动)。

为每个帧增加一个变量: `sendTime`, 表示该帧的发送时间。然后根据当前时间, 如果当前时间大于 `sendTime`, 才开始移动。

开始模拟时, 记录开始时间。

帧 0 的 `sendTime` = 开始时间

帧 1 的 `sendTime` = 开始时间 + 0.5

帧 2 的 `sendTime` = 开始时间 + 1.0

帧 3 的 `sendTime` = 开始时间 + 1.5

超时计时器: 从发送第一个未确认帧开始计时。

如果收到 ACK, 且该 ACK 确认了第一个未确认帧, 则重置计时器, 并将窗口滑动到下一个未确认帧。

当 `base` 对应的帧超时 (即发送后超过 3 秒未确认), 则重传 `base` 到 `nextSeqNum-1` 的所有帧 (即所有未确认的帧)。

`base` 初始为 0, 当帧 0 被确认, `base` 变为 1, 以此类推。当 `base=4` 时, 表示全部确认。

`int base = 0;` // 发送窗口第一个未确认帧

`int nextSeqNum = 4;` // 因为一开始发送了 0-3, 所以下一个要发送的帧序号是 4 (但这里我们不发送 4, 因为只模拟 4 个帧)

`bool ackReceived[4] = {false};` // 记录每个帧是否被确认

6. 丢帧模式:

开启后, 点击帧 (在传输过程中) 会将其标记为丢失 (`frame.lost`)

(3) 选择性重复 ARQ

特点：

- 发送方和接收方都维护
- 接收方可缓存乱序到达的帧
- 只重传丢失或损坏的帧
- 效率最高但实现复杂

预期结果：

点击"开始模拟"按钮

乱序接收场景：

开启"丢帧模式"

发送方会依次发送帧 0-3

当帧 1 出现时，点击帧 1 手动丢弃

预期结果：

接收方缓存帧 0、2、3。

信息面板显示"收到帧: 0"、"收到帧: 2"、"收到帧: 3"，但是不发送任何 ACK
等待超时

发送方超时后只重传帧 1

接收方收到帧 1 后，按序处理所有帧，并按顺序依次发送 ACK (0-3)

又比如： 手动丢弃帧 1 和帧 3

预期结果：接收方收到帧 0 和 2，但是不发送任何 ACK

等待超时

发送方超时后只重传帧 1 和帧 3

接收方独立确认每个帧

信息面板显示单独的重传信息

具体设计：

发送方：

维护一个帧数组，每个帧包含：序号、是否已发送、是否已确认、是否正在传输、位置、定时器等。发送方按顺序发送帧 0、1、2、3（每隔 0.8 秒发送一个），直到四个帧都发送出去。每个帧在发送时启动定时器（如果超时，且未确认，则重传该帧（只重传这个帧））。

接收方：

维护 expectedSeq（下一个期望交付的帧序号，初始 0）。

维护一个缓存数组(大小为 4),记录已经接收但未交付的帧(以及它们的数据)。

当收到一个帧(假设序号为 N): 如果 N 等于 `expectedSeq`, 交付该帧(标记该帧已交付), `expectedSeq++`。然后检查缓存中是否有 `expectedSeq`, 有则交付并 `expectedSeq++`, 直到缓存中没有 `expectedSeq`。然后, 为这一批连续交付的帧(从 N 到最后一个连续的帧)依次发送 ACK。如果 N 不等于 `expectedSeq`, 但 N 在`[expectedSeq, expectedSeq+3]`(窗口内), 则缓存该帧(如果之前没有缓存的话), 并且不发送 ACK。否则, 丢弃(或忽略)该帧。

超时重传: 每个帧有自己的定时器(从发送开始计时), 超时时间 3 秒。如果超时且未确认, 则重传。

发送方重传机制: 每个帧在发送时记录发送时间。在更新循环中, 检查每个已发送但未确认的帧, 如果当前时间减去发送时间大于 3 秒, 则重传该帧(重置位置, 重新开始传输)。

图形绘制:

绘制发送方、接收方区域

绘制传输中的帧(数据帧和 ACK 帧)

信息面板显示: 当前状态、已发送帧、已确认帧、接收方期望的序号、ACK 队列等。